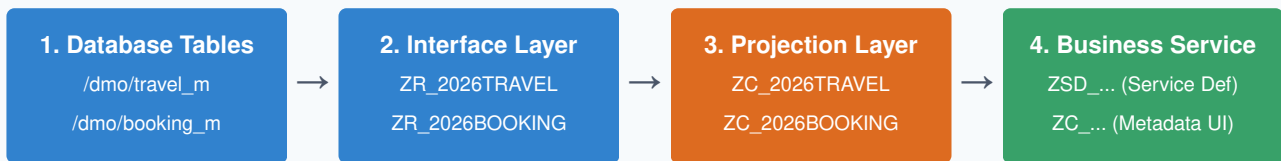

SAP RAP Architecture Guide (English)

Project: TRAVEL_BOOKING (Official Technical Guide)

Target: Complete step-by-step technical implementation artifact guide for developers and students.

ARCHITECTURE & DATA FLOW CHART



STEP 1

Database Root Table Structure (/dmo/travel_m)

Core Architecture Theory

This transparent base database table stores transactional primary travel master header entries.

```
@EndUserText.label : 'Flight Reference Scenario: Managing Travels'
define table /dmo/travel_m {
  key client      : abap.clnt not null;
  key travel_id   : /dmo/travel_id not null;
  agency_id      : /dmo/agency_id;
  customer_id    : /dmo/customer_id;
  begin_date     : /dmo/begin_date;
  end_date       : /dmo/end_date;
  @Semantics.amount.currencyCode : '/dmo/travel_m.currency_code'
  booking_fee    : /dmo/booking_fee;
  @Semantics.amount.currencyCode : '/dmo/travel_m.currency_code'
  total_price    : /dmo/total_price;
  currency_code  : /dmo/currency_code;
  description    : /dmo/description;
  overall_status : /dmo/overall_status;
  created_by     : abp_creation_user;
  created_at     : abp_creation_tstmpl;
  last_changed_by : abp_locinst_lastchange_user;
  last_changed_at : abp_locinst_lastchange_tstmpl;
}
```

STEP 2

Database Child Table Structure (/dmo/booking_m)

Core Architecture Theory

This schema serves as the one-to-many operational child entity table storing individual flight allocations.

```
@EndUserText.label : 'Flight Reference Scenario: Booking'
define table /dmo/booking_m {
  key client      : abap.clnt not null;
  key travel_id   : /dmo/travel_id not null;
  key booking_id  : /dmo/booking_id not null;
  booking_date    : /dmo/booking_date;
  customer_id     : /dmo/customer_id;
  carrier_id      : /dmo/carrier_id;
  connection_id   : /dmo/connection_id;
  flight_date     : /dmo/flight_date;
  @Semantics.amount.currencyCode : '/dmo/booking_m.currency_code'
  flight_price    : /dmo/flight_price;
  currency_code   : /dmo/currency_code;
  booking_status  : /dmo/booking_status;
  last_changed_at : abp_locinst_lastchange_tstmpl;
}
```

STEP 3

Root Interface CDS View Entity (ZR_2026TRAVEL)

Core Architecture Theory

This data model acts as the business logic root interface node establishing bidirectional relational composition parameters.

```
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'ROOT ENTITY FOR TRAVEL'
define root view entity ZR_2026TRAVEL as select from /dmo/travel_m
composition [0..*] of ZR_2026BOOKING as _Bookings
{
    key travel_id as TravelId,
    agency_id as AgencyId,
    customer_id as CustomerId,
    begin_date as BeginDate,
    end_date as EndDate,
    booking_fee as BookingFee,
    total_price as TotalPrice,
    currency_code as CurrencyCode,
    description as Description,
    overall_status as OverallStatus,
    created_by as CreatedBy,
    created_at as CreatedAt,
    last_changed_by as LastChangedBy,
    last_changed_at as LastChangedAt,
    _Bookings
}
```

STEP 4

Child Interface CDS View Entity (ZR_2026BOOKING)

Core Architecture Theory

This child interface node view sets database projection logic parameters to link back to the parent component.

```
@AbapCatalog.viewEnhancementCategory: [#NONE]
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'ENTITY FOR BOOKING'
define view entity ZR_2026BOOKING as select from /dmo/booking_m
association to parent ZR_2026TRAVEL as _Travel on $projection.TravelId = _Travel.TravelId
{
    key travel_id as TravelId,
    key booking_id as BookingId,
    booking_date as BookingDate,
    customer_id as CustomerId,
    carrier_id as CarrierId,
    connection_id as ConnectionId,
    flight_date as FlightDate,
    flight_price as FlightPrice,
    currency_code as CurrencyCode,
    booking_status as BookingStatus,
    last_changed_at as LastChangedAt,
    _Travel
}
```

STEP 5

Root Projection View Entity (ZC_2026TRAVEL)

Core Architecture Theory

This layer establishes user consumption entity filters, isolating internal operational logic from UI controls.

```
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'PROJECTION ROOT ENTITY FOR TRAVEL'
@Metadata.allowExtensions: true
define root view entity ZC_2026TRAVEL as projection on ZR_2026TRAVEL
{
    key TravelId,
    AgencyId,
    CustomerId,
    BeginDate,
    EndDate,
    BookingFee,
    TotalPrice,
    CurrencyCode,
    Description,
    OverallStatus,
    CreatedBy,
    CreatedAt,
    LastChangedBy,
    LastChangedAt,
    _Bookings : redirected to composition child ZC_2026BOOKING
}
```

STEP 6

Child Projection View Entity (ZC_2026BOOKING)

Core Architecture Theory

This consumption projection entity routes secondary children records cleanly back to their designated parent view controllers.

```
@AbapCatalog.viewEnhancementCategory: [#NONE]
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'PROJECTION VIEW ENTITY FOR BOOKING'
define view entity ZC_2026BOOKING as projection on ZR_2026BOOKING
{
    key TravelId,
    key BookingId,
    BookingDate,
    CustomerId,
    CarrierId,
    ConnectionId,
    FlightDate,
    FlightPrice,
    CurrencyCode,
    BookingStatus,
    LastChangedAt,
    _Travel : redirected to parent ZC_2026TRAVEL
}
```

STEP 7

Service Definition (ZSD_2026TRAVEL_BOOKING)

Core Architecture Theory

This technical initialization framework securely exposes final consumer endpoints to external API or UI gateways.

```
@EndUserText.label: 'SERVICE DEFINITION FOR TRAVEL_BOOKING'
define service ZSD_2026TRAVEL_BOOKING {
    expose ZC_2026TRAVEL as TRAVEL;
    expose ZC_2026BOOKING as BOOKING;
}
```

STEP 8

Metadata Extension Matrix (ZC_2026TRAVEL)

Core Architecture Theory

This dedicated layer maps logical elements directly to structural grid nodes within SAP Fiori layouts.

```
@Metadata.layer: #PARTNER
annotate entity ZC_2026TRAVEL
with
{
  @UI.lineItem: [{ position: 10 }] TravelId;
  @UI.lineItem: [{ position: 20 }] AgencyId;
  @UI.lineItem: [{ position: 30 }] CustomerId;
  @UI.lineItem: [{ position: 40 }] @UI.selectionField: [{ position: 10 }] BeginDate;
  @UI.lineItem: [{ position: 50 }] @UI.selectionField: [{ position: 20 }] EndDate;
}
```